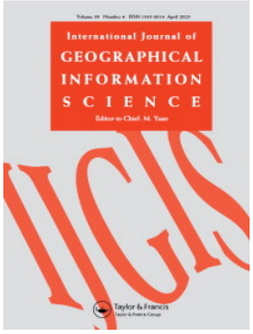


International Journal of Geographical Information Science

ISSN: 1365-8816 (Print) 1362-3087 (Online) Journal homepage: www.tandfonline.com/journals/tgis20

Medark: a map-matching error detection and rectification framework for vehicle trajectories

Zijian Wan & Somayeh Dodge

To cite this article: Zijian Wan & Somayeh Dodge (2025) Medark: a map-matching error detection and rectification framework for vehicle trajectories, International Journal of Geographical Information Science, 39:4, 872-899, DOI: 10.1080/13658816.2024.2436482

To link to this article: https://doi.org/10.1080/13658816.2024.2436482

© 2024 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.

Published online: 10 Dec 2024.

Submit your article to this journal

Article views: 1237

View related articles

View Crossmark data

Citing articles: 1 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=tgis20

INTERNATIONAL JOURNAL OF GEOGRAPHICAL INFORMATION SCIENCE
2025, VOL. 39, NO. 4, 872–899
https://doi.org/10.1080/13658816.2024.2436482

Taylor & Francis
Taylor & Francis Group

RESEARCH ARTICLE

OPEN ACCESS Check for updates

Medark: a map-matching error detection and rectification framework for vehicle trajectories

Zijian Wan and Somayeh Dodge

Department of Geography, University of California Santa Barbara, Santa Barbara, CA, USA

ABSTRACT

The widespread use of Global Navigation Satellite System (GNSS) trackers has significantly enhanced the availability of vehicle tracking data, providing researchers with critical insights into human mobility. Map matching, a key preprocessing step in movement analysis, matches vehicle tracking data to road segments but often introduces errors that can affect subsequent analyses. Existing map-matching methods, categorized into classic spatially generalizable methods and region-specific deep-learning-based methods, both have limitations. Region-specific deep learning methods, while more accurate, do not transfer well across different geographical regions. Moreover, the temporal adaptability of both approaches—their ability to handle GNSS signals of varying sampling intervals—has not been thoroughly examined. To overcome these limitations, we introduce Medark, a novel framework for detecting and rectifying errors in classic map-matching methods while preserving spatial generalizability. The proposed model is trained using a transfer-learning approach with synthetic trajectories generated in Ann Arbor and Los Angeles at various sampling intervals and a real vehicle trajectory dataset from Ann Arbor. Our experimental results validate the effectiveness of Medark. This framework can be integrated with any map-matching method to improve accuracy and produce high-quality trajectories for further analysis.

ARTICLE HISTORY
Received 2 August 2024
Accepted 26 November 2024

KEYWORDS
Map matching; error detection; long short-term memory (LSTM); transfer learning; vehicle trajectories

1. Introduction

The rapid expansion of mobile devices equipped with the Global Navigation Satellite System (GNSS), e.g. Global Positioning System (GPS), has resulted in the swift accumulation of movement data, capturing both individual movement characteristics and system-wide or aggregated patterns across road networks. This development has revolutionized movement-related research, transforming it into a discipline rich in data and heralding the advent of the era of big movement data (Graser *et al.* 2020, Nathan

CONTACT Zijian Wan zijianwan@ucsb.edu

© 2024 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.
This is an Open Access article distributed under the terms of the Creative Commons Attribution-NonCommercial-NoDerivatives License (http://creativecommons.org/licenses/by-nc-nd/4.0/), which permits non-commercial re-use, distribution, and reproduction in any medium, provided the original work is properly cited, and is not altered, transformed, or built upon in any way. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

et al. 2022). Big movement data have facilitated many trajectory-based studies in human mobility research and applications in the transportation field, such as identifying transportation modes, significant places, and human activities (Biljecki *et al.* 2013, Siła-Nowicka *et al.* 2016, Han *et al.* 2018) and analyzing traffic congestion (Kan *et al.* 2019). One crucial assumption of those applications is that the tracking points represent the true locations of the moving objects at the recorded timestamps. Nevertheless, spatial discrepancies between the recorded and real locations are prevalent, leading to a positional error ranging from 5–20 m under normal circumstances (Sharath *et al.* 2019) and even larger errors when there are other interferences such as the urban canyon phenomenon and atmospheric disturbances (Zhao *et al.* 2003). These errors can significantly impact analytical processes such as the identification of significant locations and activity places, as well as the estimation of traffic intensity on roads. That is why a preprocessing step named map matching is essential for incorporating road networks as contextual information into trajectory data, thereby transforming trajectories into more informative components, such as route choices and human daily activities.

Map matching is defined as the procedure to determine which road segment an entity is traveling on, using raw tracking data from sensors (Quddus *et al.* 2007, Newson and Krumm 2009), e.g. trajectory data collected from GNSS trackers (see Figure 1). There are many existing map-matching methods as it has been studied for several decades to support trajectory-based analyses. Map-matching methods can be divided into two categories: *spatially generalizable* methods and *spatially ungeneralizable* methods. Spatial generalizability refers to a method's transferability across different geographic locations. Classic map-matching methods, which are typically generalizable, encompass geometric, topological, and probabilistic approaches (e.g. White *et al.* 2000, Bierlaire *et al.* 2013, Li *et al.* 2023), as well as methods based on the Hidden Markov Model (HMM) (Hummel 2006, Newson and Krumm 2009). These methods, especially the one based on the HMM, are widely used in practice because not only are they straightforward and easily interpretable, but more importantly, the same algorithm can be directly applied to different trajectory datasets regardless of the regions. However, that characteristic in turn restricts the matching capability of the classic methods, especially when they encounter noisy trajectory data. To

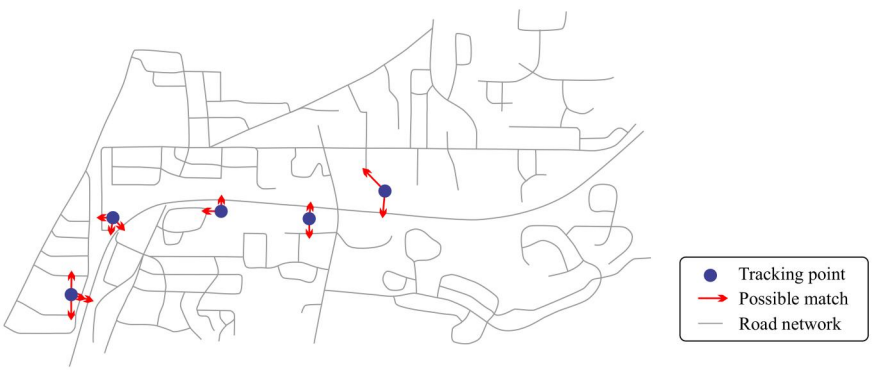


Figure 1. A schematic illustration of map matching.

improve the matching accuracy, researchers propose more advanced map-matching algorithms based on deep learning, including the ones based on the Long Short-Term Memory (LSTM) (Feng *et al.* 2020) and Transformer (Jin *et al.* 2022). They outperform classic methods on the map-matching task of trajectories with various sampling intervals. And the performance improvement is even greater when the raw trajectories contain more noise. The downside of deep-learning-based methods, however, is that they are spatially ungeneralizable, meaning that those high-performance map-matching models, once trained, are restricted to the region where the training data is collected. That inability to generalize to new regions greatly impacts their practicality. Hence, those more advanced methods are infrequently used despite the high matching accuracy they can achieve. Additionally, the temporal generalizability of map-matching methods—their ability to handle GNSS signals of varying sampling intervals—remains understudied.

Therefore, the objective of this study is to develop a map-matching framework that not only achieves high matching accuracy but also exhibits spatial and temporal generalizability. It is time to revisit the widely used, classic map-matching methods, given their inherent spatial generalizability. Herein, we propose Medark: a map-matching error detection and rectification framework, designed to detect and rectify errors in the results of classic map-matching methods while preserving spatial generalizability. The map-matching error detection model, based on the LSTM, detects errors in trajectories matched by a spatially generalizable classic map-matching method. This error detection process relies on tracking point attributes that are not characterized by spatial coordinates. Finally, the detected map-matching error points are rectified by replacing them with points of a plausible route connecting the correctly matched tracking points at both ends. The contributions of this study are summarized as follows.

- We develop an LSTM-based map-matching error detection model that exhibits spatial and temporal generalizability.
- Considering how map-matching errors occur, we design an algorithm to generate synthetic trajectories containing map-matching errors and adopt a transfer learning approach to train and evaluate the proposed map-matching error detection model.
- This map-matching error detection model, when integrated with an error-rectifying approach and a classic map-matching method, has the potential to evolve into a new spatially generalizable map-matching framework.
- Finally, we test the efficacy and performance of the proposed methods on two different road networks (Ann Arbor and Los Angeles).

2. Related work

2.1. Map-matching methods

Quddus *et al.* (2007) provide an early taxonomy of map-matching methods according to their matching mechanisms, which include geometric, topological, probabilistic, and advanced methods. The HMM-based algorithm was viewed as an advanced method at that time, but it is now considered a classic technique that is widely used. Therefore,

in this section, we start with the first three categories, then discuss the HMM-based method, and finally review the recent and more advanced map-matching techniques that leverage deep learning.

Early map-matching methods are based on geometric characteristics, e.g. point-to-point and point-to-line proximity and line-to-line similarity (Bernstein and Kornhauser 1996). These approaches are simple and intuitive and thus are highly interpretable. White *et al.* (2000) present four algorithms, from the basic point-proximity-based algorithm to the one that considers connectivity constraints and curve similarity. To ensure each matched result is routable, Greenfeld (2002) assess the similarity between a vehicle trajectory and topologically feasible routes in the road network. The similarity is assessed based on the distance and heading. Despite their intuitiveness, geometric and topological methods are sensitive to noise, leading to deviations or detours from the correct route (Singh *et al.* 2019). To account for the outliers in tracking data, Hashemi and Karimi (2016) create a weighted map-matching method based on speed, heading, the distance between observed tracking points, and estimated positional accuracy for enhanced matching accuracy.

The aforementioned methods are mostly deterministic, meaning that for each trajectory input, there is only one route output without any uncertainty evaluation. Specifically, for each tracking point in a given trajectory, there is only one target location in the road network. Since map matching often serves as a preprocessing step in trajectory-based analyses, errors introduced at this stage can propagate, resulting in unreliable outcomes. To tackle this problem, researchers propose to define a custom confident error region around a trajectory to search for multiple potential routes and assign probabilities accordingly (Bierlaire *et al.* 2013, Li *et al.* 2015). These approaches are categorized as the probabilistic map-matching methods. Compared to purely geometric and topological approaches, probabilistic methods often yield superior map-matching results, especially in urban environments. Urban features such as tall buildings, tunnels, overpasses, and the urban canyon phenomenon can disrupt or disturb GPS signals, resulting in noisy data (Le Marchand *et al.* 2009, Yang *et al.* 2021). Furthermore, the high density of users in urban environments exacerbates these disruptions, as the sheer volume of signal requests can also impact signal integrity. Probabilistic methods are particularly adept at managing these challenges, making them more effective in complex urban settings. Li *et al.* (2023) combine the geometric and probabilistic approaches. They design a robust map-matching algorithm on the basis of point-to-road proximity and trajectory-to-route similarity, incorporating a score matrix to monitor multiple matching candidates and assess their probability at each step. This method addresses noise in tracking data by setting a predetermined maximum acceptable offset. However, determining this threshold requires users to manually review each tracking dataset and balance noise suppression against matching accuracy, which restricts the flexibility of that method.

To enhance flexibility and allow for a more intelligent uncertainty assessment, researchers turn to machine learning and deep learning models. Newson and Krumm (2009) conceptualize each road segment as a distinct state within the HMM framework and the tracking data as state observations, which may be subject to noise. The HMM effectively infers transition paths (i.e. routes) from these noisy observations while

seamlessly incorporating path constraints (i.e. road topological constraints). Nevertheless, a major limitation of the HMM is its inability to consider more than the immediate previous state when predicting the current state.

Ideally, a map-matching method should consider both the previous and subsequent points of the tracking point. By accounting for these adjacent points, the method can better understand the trajectory's direction and movement pattern, resolving ambiguities when a point is near multiple road segments. Deep learning techniques excel in this area, as they integrate additional contextual information, capturing the intricate relationships between points along the trajectory and leveraging this data to make more informed and accurate decisions. Feng *et al.* (2020) introduce the DeepMM method, which employs the LSTM—a variant of the Recurrent Neural Network (RNN)—along with an encoder-decoder architecture featuring an attention mechanism. This configuration allows DeepMM to read and memorize more context information in the neighborhood before it matches a tracking point. In their experiments, DeepMM achieves significantly higher accuracy compared to the HMM-based method. Nevertheless, DeepMM is subsequently outperformed by a Transformer-based map-matching method (Jin *et al.* 2022). The Transformer, leveraging the multi-head attention mechanism (Vaswani *et al.* 2017), is designed to consider both the internal correlation between tracking points and the external correlation between the tracking data and the road network. Both deep learning models are inspired by advances in natural language processing (NLP), drawing an analogy between the map-matching problem and a language translation task. In this analogy, a trajectory—a sequence of tracking points—is viewed as a sentence in one language, while a route—a sequence of consecutive road segments—is viewed as a sentence in another language. Both trajectories and routes are characterized by their coordinates, typically represented as longitude and latitude or as x and y coordinates after projection. Through this process, deep learning models learn and memorize the road network of the region where the trajectories are collected, allowing for accurate trajectory matching within this area. However, a limitation of these models is their inability to generalize to regions they have not previously encountered. This lack of spatial generalizability limits the widespread adoption of these models as state-of-the-art map-matching toolkits.

Qu *et al.* (2023) conduct a quantitative evaluation of the matching accuracy on 14 million taxi trajectories in the central urban area of Chengdu, China. The trajectories are map-matched using the HMM algorithm. The map-matching errors are identified as tracking points that are wrongly matched to neighboring road segments or not matched since they are too far away from where they are supposed to be. Then, they evaluate the matching accuracy of the HMM method on the trajectory level and find that 89% of the trajectories are successfully matched without a single error. Velaga *et al.* (2012) assess the map-matching accuracy on the level of tracking points, which are map-matched using a topological approach. They use four tracking datasets collected in London, UK, Mumbai, India, Washington D.C., USA, and Nottingham, UK, respectively. 2,926 error points are discovered out of the total 62,887 tracking points, corresponding to a matching accuracy of 95.3%. These findings demonstrate that classic map-matching methods can correctly match the majority of trajectories and

tracking points. Thus, it is worthwhile and feasible to improve matching accuracy by detecting and rectifying the matching errors.

Given that the classic map-matching methods are spatially generalizable but less accurate, and advanced deep-learning-based techniques are highly accurate but lack spatial generalizability, it is intuitive to seek an alternative approach that achieves high accuracy while preserving spatial generalizability.

2.2. Map-matching error detection

Since directly creating a novel highly accurate map-matching method that is also spatially generalizable seems challenging, researchers turn to an alternative way. Attempts have been made to detect and rectify errors in map-matching results obtained from classic methods that are inherently spatially generalizable. For instance, Chen *et al.* (2008) introduce a map-matching validation procedure that utilizes an elliptical error region derived from the variance-covariance matrix of position estimation. A matched point is identified as an error if it lies outside this elliptical region. This method, as an early attempt to detect map-matching errors, employs a rigid approach based on the variance-covariance matrix, providing a straightforward statistical framework to identify errors. However, this method faces challenges with tracking points that exhibit a higher level of noise, as the elliptical error regions expand, leading to decreased certainty about the correctness of points within these regions. Velaga *et al.* (2012) manually examine the errors in results from a topological map-matching method and manage to enhance the method by identifying the operational environment (e.g. urban, suburban, and rural) and optimizing the weight scores with a genetic algorithm. The original topological method yields a matching accuracy of 96.5% on 5,256 tracking points in the test set, which is increased to 97.8% following enhancements. Despite involving both manual intervention and a genetic algorithm, this complicated approach achieves only a modest increase in matching accuracy. Dey *et al.* (2022) utilize a Kalman filter to estimate the noise induced by measurement errors in tracking data. Based on this estimated noise and the distance between a raw tracking point and its matched counterpart, they apply a Gaussian mixture model (GMM) to determine whether a tracking point represents a matching error. However, the GMM analyzes each tracking point independently, disregarding the spatiotemporal continuity inherent in a trajectory. Consequently, Dey *et al.* (2022) need to implement additional steps—such as connected component analysis and claw detection—to identify topological irregularities. Moreover, trajectories possess additional attributes that can be incorporated to support error detection. Therefore, there is a pressing need to develop a more intelligent map-matching error detection model. In view of the sequential nature of trajectory data and the need to leverage spatiotemporal continuity, we select the LSTM for map-matching error detection in this study. The reason is that LSTM's capacity to model temporal dependencies and process sequential data makes it particularly effective at capturing the inherent patterns in trajectories (Rossi *et al.* 2021, Wan and Dodge 2023).

3. Methodology

In this section, we introduce Medark, which features an LSTM-based model for identifying potential map-matching errors in vehicle trajectories, followed by a systematic error-rectification approach. First, we present the LSTM-based error detection model, outlining its architecture, input features, and training procedure. To minimize the need for extensive manual data labeling, a transfer-learning approach is employed. Second, we detail the error-rectification process, which includes identifying erroneous segments, selecting anchor points, and generating corrected trajectory segments based on the shortest-time path. This approach ensures the accuracy of rectified trajectories while accounting for practical factors that influence real-world travel behavior. The methodology uses the trajectory and tracking point as its building blocks, which are defined below.

Definition 1 (*Trajectory*): A trajectory $T = \{p_1, p_2, \ldots, p_n\}$ is a sequence of tracking points arranged in chronological order.

Definition 2 (*Tracking point*): A tracking point, represented as $p_i = (id_i, loc_i, t_i, A_i)$, is captured by the tracking device (e.g. a GNSS device or cellphone) with spatiotemporal information and sometimes other attributes (Laube 2014, Aghabozorgi *et al.* 2015, Wan *et al.* 2023).

3.1. Map-matching error detection model

The proposed map-matching error detection model utilizes an LSTM encoder-decoder architecture (see Figure 2), which is commonly employed in trajectory learning (Amirian *et al.* 2019, Wan *et al.* 2022). Given a map-matched trajectory, the encoder first reads the whole trajectory characterized by selected spatial attributes (e.g. speed, heading, and relative distance, etc.) and road category embedding and summarizes

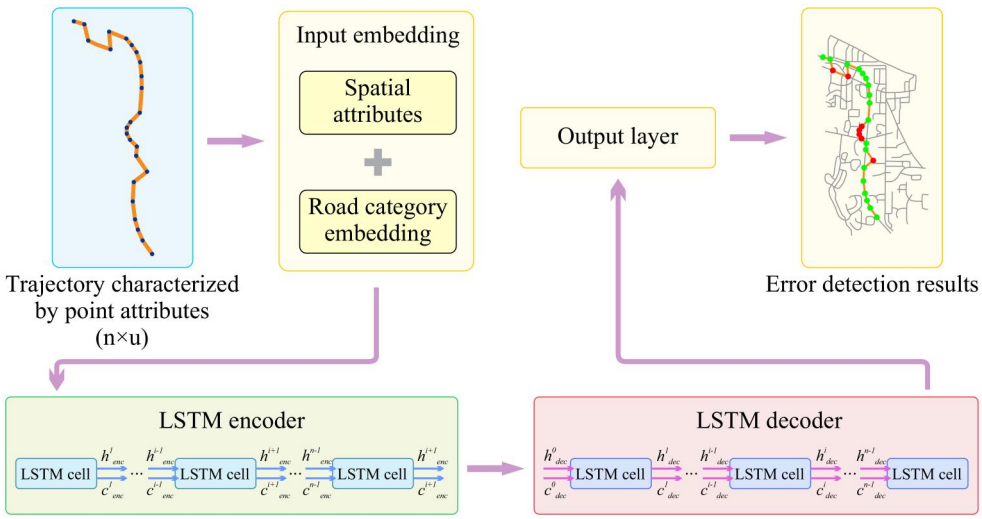


Figure 2. Architecture of the proposed map-matching error detection model. In the error detection results, the correct points are shown in green, while the erroneous ones are in red.

the information it learns into highly condensed hidden states. Utilizing these hidden states as prior information, the decoder subsequently determines whether each tracking point is correctly matched. As illustrated in the example error detection results in Figure 2, green points indicate correct matches, while the red ones signify map-matching errors.

In this study, we characterize each trajectory by its point attributes during the preprocessing phase, transforming it into a $n \times u$ matrix, where n denotes the total time steps and u is the number of attributes. These attributes detail both movement characteristics (e.g. speed and heading) and other aspects relevant to map-matching. To develop an error detection method that is spatially generalizable, we incorporate location information in a relative, rather than absolute, manner. This approach includes calculating the distance between a tracking point and its matched road segment, as well as the difference in heading between them. Although the road network data may provide only one heading per road segment—even when traffic flows in both directions—this heading difference remains informative. Specifically, a correctly matched tracking point typically exhibits a heading difference close to 0 degrees (vehicle traveling in the same direction) or 180 degrees (vehicle traveling in the opposite direction, as in the case where a road segment with a single heading attribute accommodates bidirectional traffic). This indicates proper alignment with the road segment. Conversely, a heading difference near 90 degrees often indicates a matching error. Another piece of information that is crucial in detecting map-matching errors is the road category. OpenStreetMap[1] (OSM) provides a taxonomy of roads that includes motorways, trunks, primary, and residential roads, to name a few. A common map-matching error occurs when tracking points are incorrectly assigned to adjacent road segments (Qu *et al.* 2023). For example, a vehicle on a higher-level road, such as a motorway or primary road, may exhibit some tracking points erroneously matched to nearby lower-level roads such as service or residential roads that run parallel. In this case, road category information is particularly informative, as common sense tells us that it is generally implausible for drivers to abruptly exit and then re-enter a major highway under normal circumstances. Therefore, we select these four attributes ($u = 4$) to characterize each tracking point in a trajectory.

- Change of heading, compared to its previous point.
- Heading difference between the point and the matched road.
- Distance between the point and the matched road.
- Matched road category.

Given an input trajectory, the encoder sequentially analyzes its relationship with the road network, learning the movement patterns of a vehicle within the road network. For instance, it understands that a vehicle needs to stay on the road (non-matched tracking points may appear off-road) and travel along the road (wrongly matched points may jump across neighboring roads). The obtained knowledge is compressed into a hidden state that is updated at each time step. After input embedding, the LSTM encoder processes the trajectory matrix $x_{n\times s}$ to obtain the hidden state h^t_{enc} at time step t, as defined by Equation (1).

$$(o^t_{\text{enc}}, h^t_{\text{enc}}) = \lambda_{\text{enc}}(h^{t-1}_{\text{enc}}, x^t_{n\times k}; W_{\lambda_{\text{enc}}}) \tag{1}$$

where o^t_{enc} denotes the output of the LSTM encoder and $\lambda(\cdot)$ represents an LSTM with weight W (Hochreiter and Schmidhuber 1997).

After the encoder processes the last tracking point in this input trajectory, we obtain the final hidden state of the encoder h^n_{enc}, which is highly condensed information of the trajectory summarized by the encoder. This knowledge is transferred to the decoder as the prior knowledge $h^0_{\text{dec}} = h^n_{\text{enc}}$. After initializing h^0_{dec}, the decoder together with the output layer determines whether each tracking point is correctly matched through the recurrence of Equations (2)–(3). The LSTM decoder updates its hidden state at time step t based on the hidden state h^{t-1}_{dec} and output o^{t-1} at the previous time step through Equation (2). In this way, the decoder is aware of both the overall picture and the previous output. Then, a fully connected layer transforms the LSTM output o^t into the probability of whether the tracking point at time step t is a map-matching error through Equation (3).

$$(o^t_{\text{dec}}, h^t_{\text{dec}}) = \lambda_{\text{dec}}(h^{t-1}_{\text{dec}}, o^{t-1}_{\text{dec}}; W_{\lambda_{\text{dec}}}) \tag{2}$$

$$\hat{y}^t = \phi_{\text{fc}}(o^t_{\text{dec}}; W_{\phi_{\text{fc}}}) \tag{3}$$

where o^{t-1}_{dec} denotes the LSTM output at the previous time step and $\hat{y}^t$, the output of the fully connected layer ϕ_{fc} with weight $W_{\phi_{\text{fc}}}$, denotes the map-matching error probability. A tracking point is detected as a map-matching error when $\hat{y}^t > 0.5$. Note that simplified LSTM equations are used here to illustrate the data flow in the model. For a detailed description of the LSTM mechanism, please refer to Hochreiter and Schmidhuber (1997).

3.2. Transfer-learning approach for model training

Labeling map-matching errors in real trajectories manually is a labor-intensive task. It requires superimposing trajectory data onto a road network or map and conducting manual visual analysis to determine whether each tracking point is matched correctly to the corresponding road segment. This demanding process complicates the creation of a substantial dataset of labeled trajectories. However, a high-performance deep learning model typically requires a large volume of training data to achieve the optimal result. Aiming to train a model with limited labeled data, we leverage the transfer learning approach that has two stages: pre-training and fine-tuning. We pre-train the model on a large amount of synthetic data before fine-tuning it on a small real dataset (see Figure 3). Being successfully implemented in various domains, such as computer vision (Namozov and Cho 2018, Tremblay *et al.* 2018) and trajectory modeling (Feng *et al.* 2020, Jin *et al.* 2022), transfer learning has demonstrated significant potential for training high-performance deep learning models with limited data. It is important to note that the specific transfer learning approach utilized here—employing both synthetic and real data—is just one variant of transfer learning, while it broadly encompasses any strategy that transfers knowledge from source domains to different, yet related, target domains (Weiss *et al.* 2016, Zhuang *et al.* 2021).

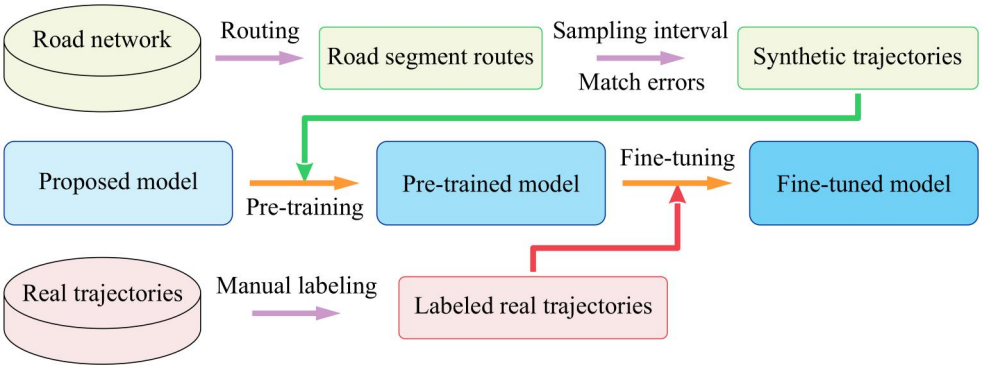


Figure 3. Transfer-learning approach for model training.

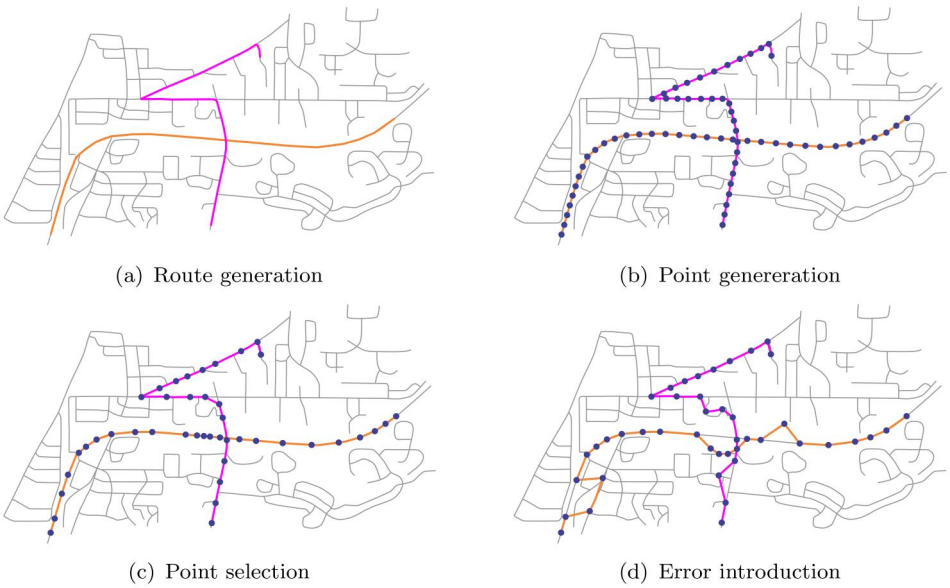


Figure 4. Generation of synthetic trajectory samples with map-matching errors. (a) Route generation. (b) Point generation. (c) Point selection. (d) Error introduction.

3.2.1. Generating synthetic trajectories

Drawing on the strategy by Jin *et al.* (2022), our synthetic trajectory generation method consists of four stages: route generation, point generation, point selection, and error introduction in map-matched tracking points (see Figure 4). Upon acquiring road network data from OSM, we generate routes between pairs of randomly selected origins and destinations using the *k*-shortest time paths (see Figure 4(a)). We opt for the *k*-shortest time paths over the *k*-shortest paths or other random route generation methods because travel time is a critical factor that people consider in route choices, although real-world route choices are much more complicated with more factors involved (Lima *et al.* 2016). Travel time is an attribute available for each road segment on OSM. To determine the *k*-shortest time paths, we rank the routes connecting the same origin and destination by travel time and select the top *k* routes. As shown in

Figure 4(b), we generate points along each route at a constant distance d_d. d_d controls the granularity of the generated points along each route. A smaller d_d results in finer granularity and more precise synthetic point positioning but requires greater computational resources. True map-matched tracking points are then selected from these points based on the sampling interval τ and speed s of each tracking point (see Figure 4(c)). To allow for variability and introduce uncertainty into the tracking data, both τ and s of each tracking point are drawn from normal distributions with means μ_τ and μ_s, respectively, as shown in Equations (4)–(5).

$$\tau \sim \mathcal{N}(\mu_\tau, \sigma_\tau^2) \tag{4}$$

$$s \sim \mathcal{N}(\mu_s, \sigma_s^2) \tag{5}$$

Here, μ_τ can be any typical sampling intervals commonly seen in a map-matching task, such as 5s or 15s. We expect μ_τ to be small, as large sampling intervals can result in a significant loss of route information, making the tracking data less suitable for map matching. As for the mean speed μ_s, we set it to the speed limit of the corresponding road segment. The variance of τ and s determines the degree of variability in the sampling intervals and speeds.

The final crucial step is to manually introduce errors into the generated tracking points, simulating the results that we typically obtain from a classic map-matching method (see Figure 4(d)). We start by choosing an error percentage err_p. When n synthetic points are generated, the number of erroneous points (map-matching errors) is $n \times err_p$. These errors fall into two categories: points erroneously matched to neighboring road segments and points unmatched due to their significant distance from where they are supposed to be (Qu *et al.* 2023). Adhering to the principle of simplicity, we distribute an equal number of errors to each category. For errors categorized as wrongly matched, we relocate the tracking point to a randomly selected neighboring road segment within a search radius of r. Observations from real-world map-matching results indicate that such errors typically involve several consecutive points, not just isolated ones. Therefore, we also move adjacent points, selecting either two or four points at random, to accompany the initial erroneous point. For errors categorized as not matched, we shift the coordinates of the tracking point by a distance d_s drawn from a normal distribution $\mathcal{N}(\mu_{d_s}, \sigma_{d_s}^2)$. The shift distances along the x and y axes are independently drawn from the distribution since the noise along the two axes is uncorrelated (Ahlgren *et al.* 2003). If the accuracy (or deviation distance) of the tracking device is known, it can be used to determine μ_{d_s}; otherwise, μ_{d_s} can be estimated by measuring the typical locational deviation—the average distance between an unmatched tracking point and its corresponding road segment. As the final step, we randomly remove a small percentage do_p of the generated tracking points to simulate potential gaps in the dataset caused by GNSS signal loss or device interference. This process mimics the random removal of information, similar to a Dropout layer in neural networks.

3.2.2. Model training

Through the above four stages, we generate a large volume of synthetic trajectory samples with known labels, which are used to pre-train the proposed map-matching

error detection model. After pre-training, the model is fine-tuned using a small set of manually labeled real trajectory samples.

For the map-matching error detection model in this study, a positive label indicates a map-matching error, while a negative label denotes a correctly matched point. In general, erroneous points constitute only a small proportion of map-matched tracking points (Qu *et al.* 2023). Consequently, the training data for the map-matching error detection model typically exhibits an imbalanced distribution of positive and negative labels. This imbalance can lead to suboptimal results if a standard binary cross-entropy loss is used during training (Rezaei-Dastjerdehei *et al.* 2020). To address this issue, we apply a weight w_p to the positive labels to counteract the imbalance. w_p is the ratio of the negative labels to the positive ones in the dataset. For the ith sample in a batch of size B, the individual loss is calculated using Equation (6). The total loss for the batch is then obtained by averaging the individual sample losses, as shown in Equation (7).

$$l_i = -\left[\omega_p y_i \cdot \ln\left(\text{Sigmoid}(\hat{y}_i)\right) + (1 - y_i) \cdot \left(1 - \text{Sigmoid}(\hat{y}_i)\right)\right] \tag{6}$$

$$\ell(\hat{y}, y) = \bar{L} = \frac{1}{B} \cdot (l_1 + l_2 + \ldots + l_B) \tag{7}$$

where $\hat{y}$ and y denote the predicted and true label vectors, respectively, and $\text{Sigmoid}(\cdot)$ represents the sigmoid function.

3.2.3. Model evaluation

At the evaluation stage, we require a human-interpretable measure to assess the model's performance in detecting map-matching errors while addressing the imbalanced distribution of labels. Accuracy is less informative in this context. For instance, in an extreme scenario where a trajectory contains 10% wrongly matched points, a model that always predicts all points as correctly matched would still achieve an accuracy of 90%, despite providing no useful information. Therefore, we adopt the kappa coefficient for model evaluation to obtain a more meaningful assessment. The kappa coefficient κ compares the observed agreement p_o against the expected agreement by chance p_e, as shown in Equations (8)–(10) (Cohen 1960, Wan *et al.* 2022). In this context, p_e takes into consideration the imbalanced distribution of labels. A false positive occurs when a correctly matched point is incorrectly identified as a map-matching error, while a false negative occurs when a map-matching error is ignored by the model.

$$\mathbb{P}_o = \frac{n_{TP} + n_{TN}}{N} \tag{8}$$

$$\mathbb{P}_e = \frac{(n_{TP} + n_{FP}) \cdot (n_{TP} + n_{FN}) + (n_{TN} + n_{FN}) \cdot (n_{TN} + n_{FP})}{N^2} \tag{9}$$

$$\kappa = \frac{\mathbb{P}_o - \mathbb{P}_e}{1 - \mathbb{P}_e} \tag{10}$$

where $n_{TP}, n_{TN}, n_{FP}, n_{FN}$ denote the number of true positives, true negatives, false positives, and false negatives, respectively, and N is the total number of all four cases.

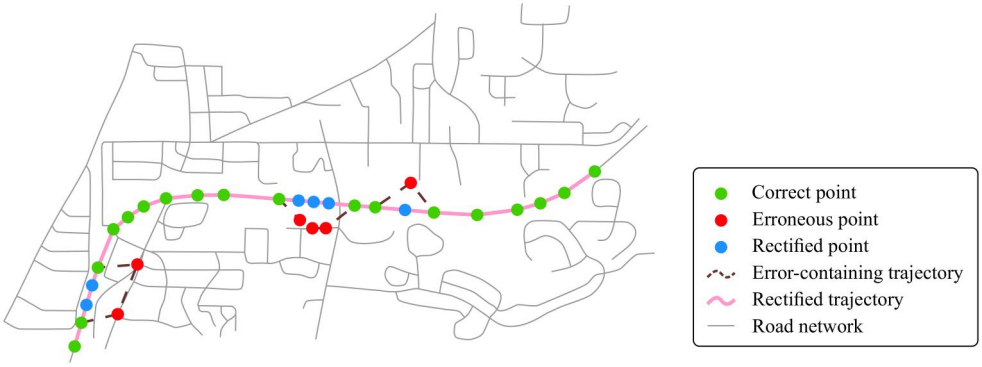


Figure 5. Rectification of detected map-matching errors.

3.3. Rectifying map-matching errors

Once map-matching errors are detected, it is crucial to rectify them to obtain correctly matched trajectories. As illustrated in Figure 5, green points represent correct matches, red points indicate detected map-matching errors, and blue points denote the rectified locations. The proposed error-rectification method is detailed in Algorithm 1. Suppose $\{p_i, p_{i+1}, \ldots, p_j\}$ is a segment of erroneous points in a trajectory $T = \{p_1, p_2, \ldots, p_n\}$. The number of erroneous points is $n_{err} = j - i + 1$. Given that the information in the erroneous points is considered lost, we rectify them based on their adjacent correct points. The first step is to identify p_{i-1} and p_{j+1} as the two anchor points before and after the erroneous segment when $1 < i < j < n$. If either of these points is an endpoint (i.e. $i = 1$ or $j = n$), the corresponding endpoint is chosen as the anchor point instead. Then, the road network nodes closest to these anchor points are selected as anchor nodes, based on which the shortest-time path is generated. We opt for the shortest-time path instead of the simple shortest path because travelers prioritize travel time over distance (Lima *et al.* 2016). The generated path is represented as a sequence of nodes $\{n_1, n_2, \ldots, n_m\}$. When the number of erroneous points is smaller than or equal to that of the nodes in the path (i.e. $n_{err} \leq m$), we can use these nodes to represent the trajectory segment. To preserve the number of tracking points, we sample a segment of n_{err} points from these m nodes while maintaining their order. Conversely, when there are more erroneous points (i.e. $n_{err} > m$), a linear interpolation is applied between each pair of adjacent nodes to generate the rectified points.

Algorithm 1 Rectifying Map-Matching Errors

Input: Trajectory $T = \{p_1, p_2, \ldots, p_n\}$, detected erroneous segment $\{p_i, p_{i+1}, \ldots, p_j\}$
Output: Rectified trajectory with corrected points
$n_{err} = j - i + 1$; // Number of erroneous points
Identify anchor points p_{i-1} and p_{j+1} (if $1 < i < j < n$);
if $i = 1$ *or* $j = n$ **then**
    Choose the corresponding endpoint as the anchor point;
end if

Select the road network nodes closest to the anchor points as anchor nodes;
Generate the shortest-time path between anchor nodes as $\{n_1, n_2, \ldots, n_m\}$;
if $n_{err} \leq m$ **then**
    Sample n_{err} points from the m nodes to represent the trajectory segment, preserving their order;
end if
else
    Apply linear interpolation between adjacent nodes to generate n_{err} rectified points;
end if
return *Rectified trajectory*

4. Experiments

To validate the proposed framework, we conduct various experiments using both synthetic and real vehicle trajectory datasets in Python. The proposed map-matching error detection model is implemented in PyTorch, with the weighted loss function (see Equations (6)–(7)) implemented using PyTorch's *BCEWithLogitsLoss*.

4.1. Datasets

4.1.1. Real trajectories

The extended vehicle energy dataset (eVED) (Zhang *et al.* 2022), an updated version of the vehicle energy dataset (VED) (Oh *et al.* 2022), is a publicly accessible vehicle trajectory dataset.[2] In addition to the fuel and energy data, eVED enriches the original dataset by incorporating GPS tracking data and traffic information such as speed limits and intersections for each trip, facilitating a broader range of studies reliant on vehicle trajectories. It includes data from 383 anonymous personal vehicles, collected in Ann Arbor, Michigan from November 1st, 2017, to November 9th, 2018, accumulating about 598,000 km over 32,940 trips. The average and median trip lengths are 18.2 km and 15.0 km, respectively. The standard GPS sampling interval in eVED is 3–5 s. However, an additional device records energy-related attributes every second. This results in multiple records of the same location to synchronize with energy data updates before the GPS tracker refreshes. For our analysis, which focuses solely on the trajectories of vehicle movement within the road network, we retain only the initial record from each GPS update, discarding subsequent energy-related data entries. A trajectory is formed by connecting adjacent tracking points from the same vehicle, provided their sampling intervals are less than the threshold of $t_s = 60$ s. In our analysis, 99.9% of the sampling intervals meet this criterion, allowing us to preserve complete vehicle trajectories to the greatest extent. Following these preprocessing steps, our dataset comprises 33,139 trajectories, encompassing over 4 million tracking points. We manually labeled 184 trajectories, selectively including those with a higher incidence of map-matching errors identified through visual analysis. As a result, the map-matching error rate within this labeled dataset is 17.7%.

4.1.2. Synthetic trajectories

We generate synthetic trajectories based on the road networks of Ann Arbor and Los Angeles, sourced from OSM, to represent distinct road network structures. Since the real vehicle trajectory is collected in Ann Arbor, we generate synthetic trajectories in Ann Arbor to pre-train the model and enhance model performance. Additionally, Los Angeles is chosen as a secondary study area due to its complex traffic patterns, characterized by a dense road network, high traffic volume, and significant travel times (Giuliano and Chakrabarti 2020). Given its considerable distance from Ann Arbor and distinct road network structure, Los Angeles provides a valuable comparison for our analysis.

As detailed in Section 3.2.1, we use a rule-based generation method. After route generation, each route is recorded with points spaced $d_d = 20$ m apart, balancing flexibility and computational efficiency. A smaller distance would yield more diverse points, but significantly increases data volume and computational load. Additionally, $d_d = 20$ m approximates the positional deviation of most GNSS devices. We create six datasets for each region with sampling intervals drawn from three Normal distributions $\mathcal{N}(\mu_\tau, \sigma_\tau^2)$, where $\mu_T \in \{2, 5, 10, 30, 60, 120\}$ and $\sigma_T^2 = \frac{\mu_T}{5}$. The speed of each generated tracking point is drawn from a Normal distribution $\mathcal{N}(\mu_s, \sigma_s^2)$ where μ_s matches the speed limit of the road segment and $\sigma_s^2 = \frac{\mu_s}{2}$. The map-matching error percentage in the synthetic trajectories err_p is set to 20%, mirroring the error rate in the labeled real dataset. These errors are evenly split between wrongly matched errors–where tracking points and their adjacent points (either two or four, chosen with equal probability) are relocated to a nearby road segment within a 100 m radius–and unmatched errors, created by shifting the coordinates of a tracking point by a distance d_s drawn from a Normal distribution $\mathcal{N}(100, 30)$. Additionally, we randomly omit a small percentage ($do_p = 0.1$) of tracking points to simulate signal loss or GNSS device interference. Ultimately, this process generates over 32,000 trajectories and more than 5 million tracking points in each region.

4.1.3. Trajectory samples

Since the sequence length (number of data points) affects the performance of LSTM-based models (Kim *et al.* 2020, Wan and Dodge 2023), we generate trajectory samples from both real and synthetic data, ensuring each sample contains between 10 and 100 tracking points. This range mimics how human observers identify map-matching errors. When superimposing trajectories on a road network, human observers require several neighboring tracking points for accurate judgment, while examining too many points simultaneously can lead to confusion rather than improved accuracy. Table 1 summarizes the numbers of tracking points and trajectory sample lengths for each dataset. The sampling interval for each dataset is indicated in parentheses. 'AA' refers to synthetic trajectories generated in Ann Arbor, while 'LA' denotes those generated in Los Angeles. In each region, synthetic samples with larger sampling intervals result in longer lengths but fewer tracking points. Synthetic samples generated in Los Angeles are longer than those in Ann Arbor, despite having a similar number of tracking points. This is because Los Angeles has more freeways, leading to vehicles traveling at faster speeds.

Table 1. Trajectory sample lengths.

Dataset	# of tracking points mean ± standard deviation.	Length (km) mean ± standard deviation.
eVED real (5 s)	69 ± 32	3.1 ± 2.1
AA synthetic (2 s)	94 ± 19	4.7 ± 2.2
AA synthetic (5 s)	74 ± 29	7.1 ± 3.9
AA synthetic (10 s)	48 ± 23	7.9 ± 4.7
AA synthetic (30 s)	23 ± 6	9.9 ± 3.8
AA synthetic (60 s)	13 ± 3	10.4 ± 3.3
AA synthetic (120 s)	11 ± 1	12.9 ± 3.4
LA synthetic (2 s)	95 ± 16	5.6 ± 1.8
LA synthetic (5 s)	88 ± 24	10.5 ± 4.3
LA synthetic (10 s)	74 ± 30	16.4 ± 8.3
LA synthetic (30 s)	38 ± 18	23.1 ± 12.6
LA synthetic (60 s)	23 ± 7	27.8 ± 11.7
LA synthetic (120 s)	15 ± 4	31.7 ± 10.0

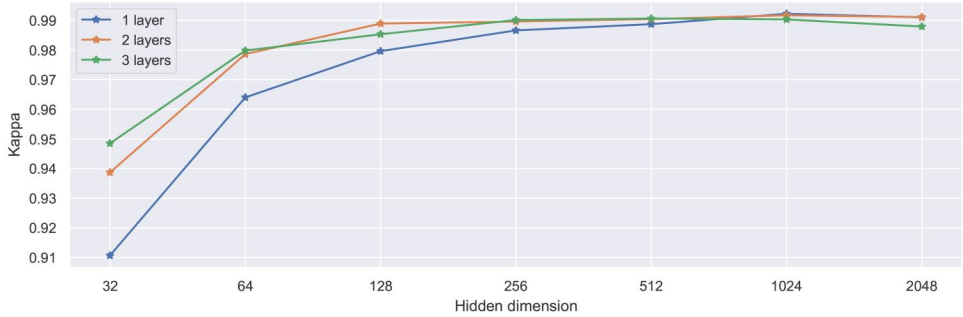
4.2. Map-matching error detection

4.2.1. Sensitivity analysis and model comparison

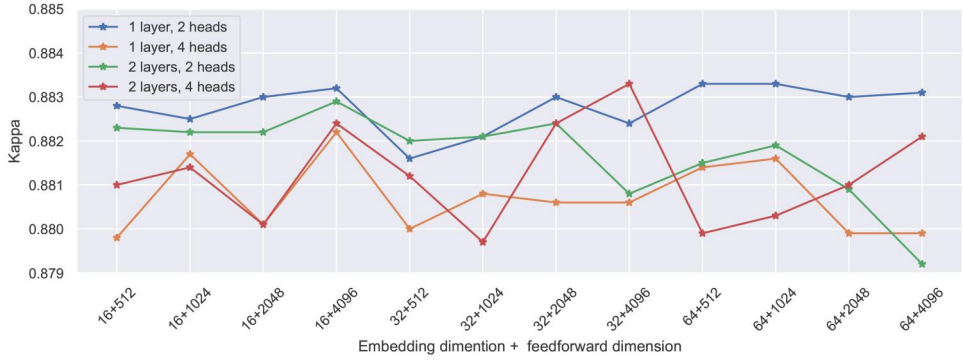
The error detection model is the most crucial component of the proposed Medark framework. To optimize its performance, we conduct a sensitivity analysis to identify the most effective hyperparameters, thereby enhancing the model's accuracy and reliability. For this analysis, we utilize a manually labeled subset of eVED, the real trajectory dataset, to train and evaluate each model. We opt for manually labeled data over synthetic data to more accurately gauge real-world performance. From the dataset comprising 9,422 sample trajectories, we randomly choose 6,595 samples for training (lengths ranging from 3065.1±2149.2 m), 1,894 for validation (lengths ranging from 3007.1±2102.5 m), and 933 for testing (lengths ranging from 3005.8±2059.3 m), maintaining a ratio of 0.7:0.2:0.1.

The proposed LSTM-based map-matching error detection model incorporates two crucial hyperparameters that affect model performance: the number of LSTM layers in both the encoder and decoder and the dimension of each hidden state. Following previous practices in sensitivity analysis (Gupta *et al.* 2018, Wan and Dodge 2023), we experiment with the number of layers ranging from one to three, and hidden dimensions from 32 to 2048. Figure 6(a) visualizes the model performance, specifically, the kappa coefficient achieved on the validation set, in relation to its hyperparameter settings. The best map-matching error detection model is obtained with one layer of LSTM and a hidden dimension of 1024 (kappa = 0.99).

Given the demonstrated superiority of the Transformer model in both NLP tasks and trajectory-based analyses (Jin *et al.* 2022, Liang *et al.* 2023), we also compare the performance of our LSTM-based method with a Transformer-based method for building the error detection model. Like the LSTM-based model, the Transformer-based model features an encoder-decoder architecture. The Transformer-based model involves tuning four key hyperparameters: the number of encoder and decoder layers, the number of heads in multi-head attention (Vaswani *et al.* 2017), the embedding dimension, and the dimension of the feedforward network. Figure 6(b) presents the results of the sensitivity analysis for the Transformer-based model, revealing that while hyperparameters impact the Transformer model less significantly than the LSTM



(a) LSTM hyperparameters



(b) Transformer hyperparameters

Figure 6. Hyperparameter setting and model comparison.

model—as indicated by a narrower range of kappa values—the results display greater irregularity. In our experiments, the Transformer model trains approximately three times as fast in each epoch as the LSTM model and converges in half of the epochs required by the LSTM model. However, the efficiency gains are marginal due to our relatively small training dataset. Nevertheless, the LSTM-based map-matching error detection model outperforms the Transformer-based model in the kappa coefficient, indicating greater effectiveness in detecting map-matching errors. This superiority may stem from the relative simplicity of the map-matching error detection task, which does not necessitate the sophistication of the Transformer model.

4.2.2. Model training and evaluation

With the optimal hyperparameter setting (i.e. one layer of LSTM and a hidden dimension of 1024), we perform training and evaluation of the proposed map-matching error detection model. To assess the spatial and temporal generalizability of the proposed model in detail, we conduct a series of comparative analysis experiments involving pre-training and fine-tuning, as shown in Table 2. For instance, the training data for the second experiment is 'E1 + AA synthetic (5 s)', meaning that the results in this row are achieved by further training the model from experiment E1 with synthetic trajectory samples generated in Ann Arbor with an average sampling interval of 5 s.

Table 2. Training and evaluation of the proposed map-matching error detection model[a].

Kappa on test data

Expt. #	Training data	AA synthetic (2 s)	AA synthetic (5 s)	AA synthetic (10 s)	AA synthetic (30 s)	AA synthetic (60 s)	AA synthetic (120 s)	eVED real (5 s)	LA synthetic (2 s)	LA synthetic (5 s)	LA synthetic (10 s)	LA synthetic (30 s)	LA synthetic (60 s)	LA synthetic (120 s)
E1	AA synthetic (2 s)	**0.98**	0.79	0.69	0.57	0.49	0.57	0.37	0.83	0.81	0.71	0.61	0.48	0.47
E2	E1 + AA synthetic (5 s)	0.90	**0.97**	0.85	0.77	0.75	0.77	0.38	0.83	0.88	0.82	0.78	0.69	0.70
E3	E2 + AA synthetic (10 s)	0.84	0.90	**0.95**	0.86	0.86	0.85	0.33	0.79	0.87	0.84	**0.84**	0.80	0.83
E4	E3 + AA synthetic (30 s)	0.75	0.81	0.88	**0.92**	0.98	0.98	0.25	0.73	0.80	0.77	**0.84**	0.89	0.97
E5	E4 + AA synthetic (60 s)	0.73	0.79	0.84	0.86	**0.99**	**1.00**	0.29	0.89	0.78	0.76	0.81	0.98	**0.99**
E6	E5 + AA synthetic (120 s)	0.73	0.78	0.84	0.86	**0.99**	0.99	0.27	0.89	0.78	0.76	0.80	0.97	**0.99**
E7	E2 + eVED real (5 s)	0.69	0.75	0.59	0.40	0.28	0.30	**0.99**	0.60	0.75	0.66	0.53	0.38	0.37
E8	E7 + LA synthetic (2 s)	0.57	0.57	0.58	0.66	0.99	0.99	0.40	**1.00**	0.60	0.57	0.58	**0.97**	0.97
E9	E8 + LA synthetic (5 s)	0.86	0.87	0.84	0.76	0.68	0.65	0.38	0.83	**0.98**	**0.87**	0.79	0.61	0.54

[a]The highest kappa coefficient for each test dataset (column) is highlighted in bold.

In experiment E1, we begin by pre-training the proposed model on a synthetic dataset with 2 s sampling intervals, generated in Ann Arbor. As with the previous procedure, all datasets in this section are split into training, validation, and test sets in a ratio of 0.7:0.2:0.1. The model that achieves the highest kappa coefficient on the validation set is selected as the best model from experiment E1 and is subsequently evaluated on various test sets. As expected, this model achieves the highest kappa coefficient on the test set from the same dataset, as its data is the most similar to the training data. As the sampling interval of the Ann Arbor synthetic samples increases to 5 s, 10 s, 30 s, and 60 s the achieved kappa decreases. Interestingly, the kappa coefficients for the first two Los Angeles synthetic datasets are close, with the highest kappa achieved on the 5 s average sampling interval among all Los Angeles synthetic datasets. This suggests that regional factors, such as varying road network structures and traffic patterns (e.g. higher vehicle speeds due to more freeways in Los Angeles), may have a greater impact on performance than the difference between 2 s and 5 s sampling intervals.

Then, we further train this model on Ann Arbor synthetic samples with a 5 s average sampling interval. In experiment E2, the best model again achieves the highest kappa on the test set from the same dataset, with lower kappa coefficients on Ann Arbor synthetic samples with other sampling intervals. This pattern is also observed when the model is applied to synthetic trajectory samples generated in Los Angeles. Since the training data has an average sampling interval of 5 s, the highest kappa 0.88 is achieved with Los Angeles synthetic samples with a 5 s average sampling interval. Nonetheless, due to differences in the road network structure and traffic patterns between Ann Arbor and Los Angeles, the highest kappa achieved by this model in Los Angeles is lower than that in Ann Arbor.

Next, the model from experiment E2 is further fine-tuned using Ann Arbor synthetic samples with 10 s, 30 s, 60 s, and 120 s sampling intervals in experiments E3–E6. Although the kappa coefficient on the test set from the same dataset is typically the highest, we observe that the kappa values for Ann Arbor synthetic samples with 60 s and 120 s sampling intervals, as well as those for LA synthetic samples with the same intervals, are close. This suggests that the difference between 60 s and 120 s average sampling intervals is minimal. More importantly, at these larger sampling intervals, the regional influence of varying road network structures becomes much less significant compared to smaller intervals.

Notably, the models in the experiments E1–E6 do not perform well on the eVED real trajectory samples, indicating conspicuous differences between real and synthetic map-matched trajectories. Consequently, we fine-tune the proposed model on the training set of eVED real trajectory samples, selecting the model from experiment E2 as the pre-trained model due to the matching sampling interval with the real dataset. This model achieves a kappa coefficient of 0.99 on the test set of eVED real samples, indicating that the knowledge learned from synthetic trajectories can be effectively transferred to real trajectories. This kappa coefficient is the same as the result (0.99) obtained in Section 4.2.1. However, it is important to note that the model used in the sensitivity analysis is trained on a small dataset, limiting its generalizability for map-matching error detection. In contrast, the model in experiment E7 is pre-trained on a large volume of synthetic trajectory samples and then fine-tuned on a smaller set of real samples, enhancing its applicability.

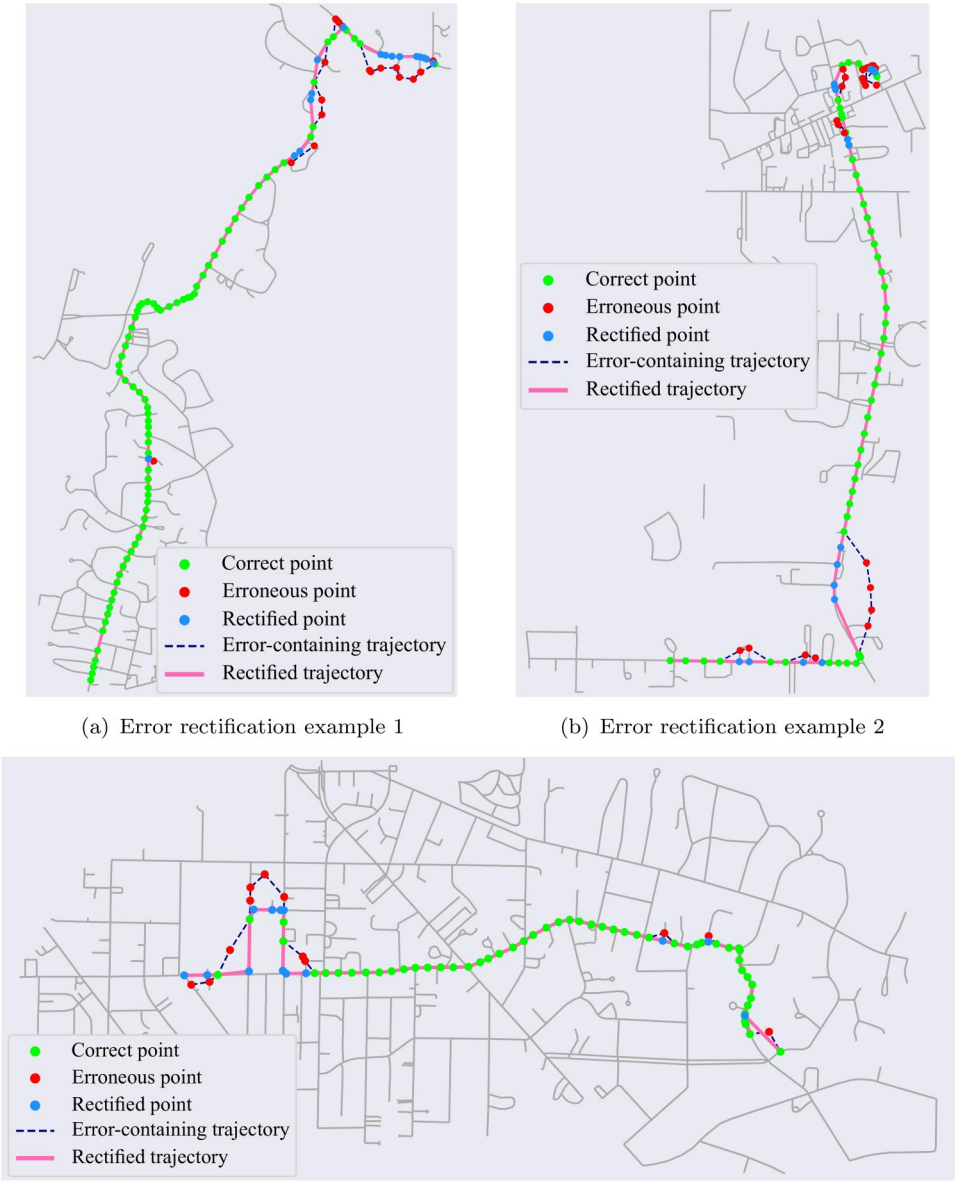
To assess whether the knowledge can be transferred to other regions, we further train the model from Experiment E7 using Los Angeles synthetic samples with 2 s and 5 s average sampling intervals in Experiments E8–E9. Both experiments confirm that map-matching error detection knowledge can be successfully transferred across regions, as demonstrated by the high kappa coefficients achieved on the test sets of the corresponding Los Angeles synthetic samples. Interestingly, the best model from Experiment E8 achieves high Kappa coefficients on both Ann Arbor and Los Angeles synthetic samples with 60 s and 120 s average sampling intervals, but a low Kappa on Ann Arbor synthetic samples with a 2 s average sampling interval, despite the training data having the same interval. In almost all experiments, the highest Kappa coefficient on each test dataset (highlighted in bold) is consistently achieved when the model is trained or fine-tuned on the corresponding training data, emphasizing the importance of fine-tuning.

4.3. Map-matching error rectification

Figure 7 visualizes four examples of map-matching error rectification results. These examples include both categories of map-matching errors: wrongly matched points and unmatched points, both displayed as erroneous points in the figure (shown in red). The vast majority of these errors are effectively detected by the proposed error detection model and subsequently rectified, resulting in rectified points (shown in blue) and a more accurate trajectory (shown in magenta). The proposed method is particularly suitable for addressing sparse errors, where only a few erroneous points are interspersed among many correct ones, which is the most common scenario for map-matching errors. As shown in the left portion of Figure 7(c), even in a dense road network where half of the tracking points in that segment are wrongly matched, the proposed framework can still produce the correct trajectory. However, the error rectification approach may not accurately reflect vehicle movement patterns, especially when the error segment is long. For instance, in Figure 7(a,b), the method becomes less effective when the number of erroneous points or the length of the error segment is relatively large since it does not consider the movement patterns indicated by both the error segment and the correct segments of the trajectory.

The rightmost part of Figure 7(c) illustrates a case where an erroneous point is successfully detected but not properly rectified. A zoomed-in evaluation is provided in Figure 8. The erroneous point (in red) is accurately detected by the proposed map-matching error detection model, and the correct target road segment (in orange) is identified. However, since the method does not account for existing points on this road segment, the rectified point is incorrectly placed (indicated by the purple arrow). While the erroneous point is the second tracking point in the trajectory, the rectified point is mistakenly positioned between the fifth and sixth points. Properly assigning the point (shown by the light blue arrow) requires subdividing the target road segment into smaller sections (represented by the dashed light blue line) to account for the existing points.

To evaluate the misplacement of rectified points, we randomly select 200 samples from the eVED real vehicle trajectory dataset, manually identify the misplacement errors through visual analysis, and calculate the misplacement rate. These 200 sample



(a) Error rectification example 1

(b) Error rectification example 2

(c) Error rectification example 3

Figure 7. Map-matching error rectification results for the eVED real trajectory dataset. The road network within a 500 m radius of each trajectory is displayed.

trajectories contain a total of 19,123 tracking points, of which 16,450 are correctly matched and 2,673 are map-matching errors, resulting in a map-matching error rate of 14.0%. Among these errors, only 184 rectified points are incorrectly placed ($184/2673 = 6.9\%$), demonstrating that the vast majority of detected errors are effectively rectified. By detecting and correcting map-matching errors, the proposed framework significantly improves the overall quality of map-matched trajectories.

(d) Error rectification example 4

Figure 7. Continued

5. Discussion

In this paper, we introduce Medark, a framework designed to enhance map-matching accuracy and produce high-quality trajectories for further analysis by detecting and rectifying map-matching errors. The proposed map-matching error detection model leverages four key attributes to capture vehicle movement patterns and interactions with the road network: (1) change in heading compared to the previous point, (2) heading difference between the point and the matched road, (3) distance between the point and the matched road, and (4) matched road category. These attributes are straightforward and intuitive, closely aligning with the criteria human observers use to assess whether tracking points in a trajectory are correctly matched. Our experiments demonstrate the effectiveness of these attributes, with the kappa statistic achieving a value of 0.99 on real vehicle trajectories from the eVED dataset, approaching the ideal score of 1. Given the marginal improvements in performance from adding more attributes, coupled with the increase in computational cost, we opt not to include additional features. This is especially pertinent since map matching typically serves as a preprocessing step in human mobility analysis, where excessive time and computational resources may not be justified. However, for more complex road networks and vehicle tracking data, the proposed model is flexible and allows users to select additional attributes to suit their specific requirements.

Importantly, the attributes used in this study encode locational information in a relative rather than absolute manner. This lays a solid foundation for the spatial generalizability of the model. Anomalies in the patterns reflected by these attributes typically suggest potential map-matching errors. However, it is important to recognize that anomalies identified by the model may sometimes represent unusual yet correctly matched patterns. These instances require additional investigation in future studies to determine whether the detected anomalies are genuine map-matching errors or revealed abnormal patterns that are accurately mapped.

In this study, we employ a transfer-learning approach to train our model, reducing the need for extensive manual annotation of trajectory samples. A large dataset is

crucial for training deep learning models to have generalizability and avoid overfitting. Our error detection model is first pre-trained on a large number of synthetic trajectories before being fine-tuned on a small set of real ones. We generate these synthetic trajectories using a rule-based method, which is simple, interpretable, and capable of producing high volumes of trajectory data across diverse regions and sampling intervals. This allows us to pre-train the proposed error detection model and evaluate its spatial and temporal generalizability. However, while rule-based methods are simpler and more interpretable than data-driven approaches, they may lack accuracy, as evidenced by the lower kappa coefficient observed when applying the pre-trained model directly to real trajectories without fine-tuning. This indicates that some complex patterns in real trajectories are not captured by the synthetic ones. Nonetheless, once fine-tuned with a small real trajectory dataset, the model achieves satisfactory performance. Given our primary goal at this stage is to generate synthetic data for pre-training, the complexity and additional training requirements of data-driven synthetic trajectory generation methods are not justified. Instead, we have strategically chosen a rule-based generation approach to reduce reliance on extensive real trajectory data and to avoid the burden of additional model training. It is necessary to note that, at this stage, any simulation or generative model can be used to create the synthetic trajectory dataset. Recognizing that more realistic trajectories can enhance model performance, future work will focus on refining this generation method to produce more realistic trajectories while maintaining its simplicity and interoperability.

Additionally, it is crucial that the rules and parameters for generating synthetic trajectories are tailored to the study area. For example, the speed distribution described in Equation (5) allows some synthetic tracking points to exceed the official speed limit. While this is not uncommon in the United States, there are regions where speed limits are enforced much more strictly. For the proposed framework to be applicable in such regions, the speed distribution may need to be adjusted, either by adopting a truncated normal distribution or using a mean speed μ_s lower than the speed limit.

Leveraging the transfer-learning approach, we evaluate the proposed map-matching error detection model using real vehicle trajectories from the eVED dataset and synthetic trajectories generated in Ann Arbor and Los Angeles, across a range of sampling intervals. The combination of pre-training and multiple fine-tuning steps highlights the model's spatial and temporal generalizability. Our results indicate that both regional factors—such as varying road network structures and traffic patterns (e.g. higher vehicle speeds due to more freeways in Los Angeles)—and sampling intervals significantly impact the model's performance. However, the relative importance of these factors varies by scenario, and further investigation is needed in future work to better understand their respective influences.

The proposed error-rectification method effectively corrects the majority of detected map-matching errors. However, on a few occasions, the rectified point may be incorrectly positioned (see Figure 8), as the method does not account for the relative positions of existing points on the target road segment. In future studies, we plan to subdivide road segments to achieve more precise point placement. It is important to note, however, that map matching typically serves as a preprocessing step in

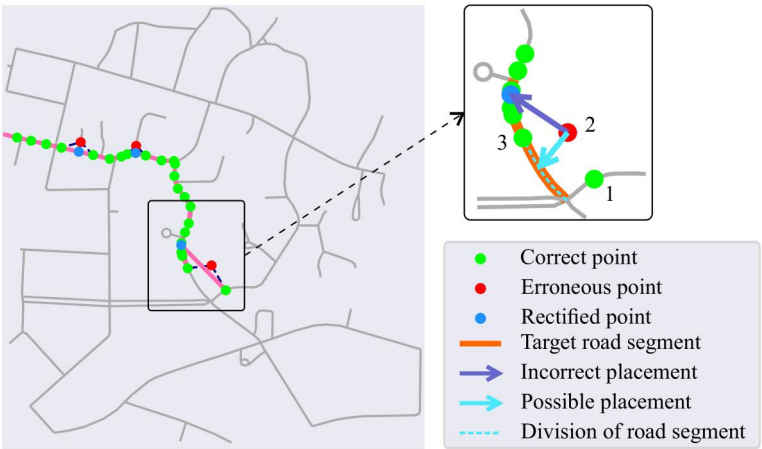


Figure 8. Rectification result evaluation. Zoom in on the rightmost portion of error rectification example 3.

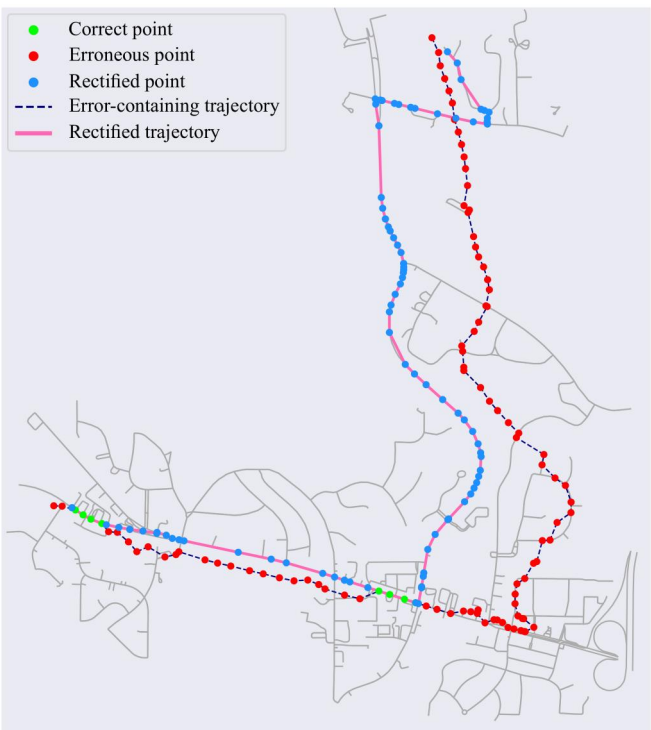


Figure 9. Example trajectory with a large portion of map-matching errors.

human mobility analysis. When vehicle trajectories are converted to routes after map matching, such misplacements of rectified points no longer occur, as routes are represented at the road segment level and are unaffected by slight inaccuracies in point placement on the correct road segment.

In rare instances, trajectories recorded with significant noise can lead to a substantial number of map-matching errors (see Figure 9 for example). Such errors may stem from systematic positioning or recording issues with the GNSS tracker. While the proposed error detection model can successfully identify these errors, the error rectification method may not perform as well. For instance, Figure 9 illustrates a systematic spatial deviation, especially in the vertical segment of this trajectory. The proposed error-rectification method successfully rectifies the majority of erroneous points based on the shortest-time path between anchor points. However, this success is partly due to the lack of a dense and complicated road network within the search radius of those erroneous points. In such complex scenarios, the method would fail because the segment lacks correctly matched tracking points, preventing the identification of necessary anchor points for deriving the shortest path. In this study, we categorize trajectories with extensive errors as outliers that should not be utilized to support further trajectory-based analyses. Nevertheless, a human observer can easily deduce that the vertical segment should align with the adjacent road on the left due to geometric similarities. This underscores the need to develop a more intelligent and robust error rectification method in the future. Additionally, the spatiotemporal movement patterns contained in the map-matching errors are not restored, as the information in the erroneous points is considered lost in this study. When there is a long sequence of consecutive erroneous points, it should be treated as a gap in the trajectory. Addressing such problems requires more advanced gap-interpolation and generative trajectory modeling methods (e.g. Long 2016, Wan and Dodge 2023).

6. Conclusion

This study introduces Medark: a map-matching error detection and rectification framework, designed to enhance map-matching accuracy. Rather than developing a new map-matching method from scratch, we present an LSTM-based model for detecting errors in existing methods. This model utilizes four key attributes to identify discrepancies: change of heading, heading difference between the point and its matched road, distance from the point to the matched road, and the road category. These attributes encode locational information in a relative manner, which not only ensures spatial generalizability but also mimics the decision-making process of human observers assessing the accuracy of trajectory tracking points. To reduce the extensive manual annotation required for trajectory samples, we adopt a transfer-learning approach. The model is initially pre-trained on a large dataset of synthetic trajectories and subsequently fine-tuned with a smaller set of real data. Once map-matching errors are detected, they are rectified based on the shortest-time path between adjacent, correctly matched points. Experimental results using both synthetic trajectories in Ann Arbor and Los Angeles and a real vehicle trajectory dataset from Ann Arbor confirm the effectiveness of our framework. The evaluation demonstrates that the model possesses spatiotemporal generalizability. However, its performance in new regions or with different sampling intervals can be further enhanced by fine-tuning the model with relevant data. This error detection and rectification framework can be integrated with any map-matching method to improve accuracy and produce high-quality trajectories for further analysis.

Disclosure statement

The authors report there are no competing interests to declare.

Notes

1. https://www.openstreetmap.org/, accessed on May 5th, 2024.
2. https://github.com/zhangsl2013/eVED, accessed on April 29th, 2024.

Funding

The authors gratefully acknowledge the support from the National Science Foundation through award BCS # 2043202.

Notes on contributors

Zijian Wan is a Ph.D. candidate in Geography (Geographic Information Science) and a member of the MOVE Lab at the University of California, Santa Barbara. His research interests include geospatial data science and GeoAI modeling of human mobility and animal movement ecology.

Somayeh Dodge is an associate professor of spatial data science and the co-associate director of the Center for Spatial Studies and Data Science at the University of California, Santa Barbara. Her research focuses on developing data analytics, knowledge discovery, modeling, simulation, and visualization techniques to study movement and spatiotemporal processes in social and ecological systems.

Data and codes availability statement

The extended vehicle energy dataset (eVED) (Zhang *et al.* 2022) is a publicly accessible vehicle trajectory dataset available at: https://github.com/zhangsl2013/eVED. The OpenStreetMap (OSM) road network graphs of Ann Arbor and Los Angeles are obtained using the *OSMnx* Python package (Boeing 2024). The code and the labeled vehicle trajectory map-matching sample data used in this paper are available at: https://doi.org/10.5281/zenodo.14225664.

References

Aghabozorgi, S., Seyed Shirkhorshidi, A., and Ying Wah, T., 2015. Time-series clustering – a decade review. *Information Systems*, 53, 16–38.

Ahlgren, P., Jarneving, B., and Rousseau, R., 2003. Requirements for a cocitation similarity measure, with special reference to Pearson's correlation coefficient. *Journal of the American Society for Information Science and Technology*, 54 (6), 550–560.

Amirian, J., *et al.*, 2019. Social ways: learning multi-modal distributions of pedestrian trajectories with GANs. *IEEE/CVF conference on computer vision and pattern recognition workshops*.

Bernstein, D., and Kornhauser, A., 1996. *An introduction to map matching for personal navigation assistants*. Newark: New Jersey TIDE Center. www.njtide.org.

Bierlaire, M., Chen, J., and Newman, J., 2013. A probabilistic map matching method for smartphone GPS data. *Transportation Research Part C: Emerging Technologies*, 26, 78–98.

Biljecki, F., Ledoux, H., and van Oosterom, P., 2013. Transportation mode-based segmentation and classification of movement trajectories. *International Journal of Geographical Information Science*, 27 (2), 385–407.

Boeing, G., 2024. *Modeling and analyzing urban networks and amenities with OSMnx*. https://geoffboeing.com/publications/osmnx-paper/.

Chen, W., *et al.*, 2008. An integrated map-match algorithm with position feedback and shape-based mismatch detection and correction. *Journal of Intelligent Transportation Systems*, 12 (4), 168–175.

Cohen, J., 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20 (1), 37–46.

Dey, S., Tomko, M., and Winter, S., 2022. Map-matching error identification in the absence of ground truth. *ISPRS International Journal of Geo-Information*, 11 (11), 538.

Feng, J., *et al.*, 2020. DeepMM: deep learning based map matching with data augmentation. *IEEE Transactions on Mobile Computing*, 21 (7), 1–1.

Giuliano, G., and Chakrabarti, S., 2020. Analyzing intra-metropolitan variation in highway traffic performance in Los Angeles using archived real-time data. *Transportation Planning and Technology*, 43 (8), 751–770.

Graser, A., Widhalm, P., and Dragaschnig, M., 2020. The M^3 massive movement model: a distributed incrementally updatable solution for big movement data exploration. *International Journal of Geographical Information Science*, 34 (12), 2517–2540.

Greenfeld, J.S., 2002. Matching GPS observations to locations on a digital map. *Transportation Research Board 81st Annual Meeting*, 22, 576–582. https://www.researchgate.net/publication/246773761.

Gupta, A., *et al.*, 2018. Social GAN: socially acceptable trajectories with generative adversarial networks. *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2255–2264.

Han, Z., *et al.*, 2018. Reconstructing individual activity trajectories by hidden semi-Markov model. *2018 26th international conference on geoinformatics*. IEEE, 1–6.

Hashemi, M., and Karimi, H.A., 2016. A weight-based map-matching algorithm for vehicle navigation in complex urban networks. *Journal of Intelligent Transportation Systems*, 20 (6), 573–590.

Hochreiter, S., and Schmidhuber, J., 1997. Long short-term memory. *Neural Computation*, 9 (8), 1735–1780.

Hummel, B., 2006. Map matching for vehicle guidance. *In*: R. Billen, E. Joao, and D. Forrest, eds. *Dynamic and mobile GIS*. 1st ed. Boca Raton: CRC Press, 211–222.

Jin, Z., *et al.*, 2022. Transformer-based map-matching model with limited labeled data using transfer-learning approach. *Transportation Research Part C: Emerging Technologies*, 140, 103668.

Kan, Z., *et al.*, 2019. Traffic congestion analysis at the turn level using Taxis' GPS trajectory data. *Computers, Environment and Urban Systems*, 74 (April 2018), 229–243.

Kim, S.J., *et al.*, 2020. State of health estimation of Li-Ion batteries using multi-input LSTM with optimal sequence length. *2020 IEEE 29th international symposium on industrial electronics (ISIE)*, 6. 1336–1341.

Laube, P., 2014. *Computational movement analysis*.

Le Marchand, O., *et al.*, 2009. ATTI dell'Istituto Italiano di Navigazione. *ATTI Dell'Istituto Italiano di Navigazione*, 189, 77–88. https://hal.science/hal-00445114.

Li, W., *et al.*, 2023. A robust map matching method by considering memorized multiple matching candidates. *Theoretical Computer Science*, 941, 104–120.

Li, H., Kulik, L., and Ramamohanarao, K., 2015. Robust inferences of travel paths from GPS trajectories. *International Journal of Geographical Information Science*, 29 (12), 2194–2222.

Lima, A., *et al.*, 2016. Understanding individual routing behaviour. *Journal of the Royal Society Interface*, 13 (116), 20160021.

Liang, Y., *et al.*, 2023. Traffic state prediction based on spatio-temporal graph transformer network. *2023 6th international conference on algorithms, computing and artificial intelligence*. Association for Computing Machinery, 222–227.

Long, J.A., 2016. Kinematic interpolation of movement data Kinematic interpolation of movement data. *International Journal of Geographical Information Science*, 30 (5), 854–868.

Namozov, A., and Cho, Y.I., 2018. An efficient deep learning algorithm for fire and smoke detection with limited data. *Advances in Electrical and Computer Engineering*, 18 (4), 121–128.

Nathan, R., *et al.*, 2022. Big-data approaches lead to an increased understanding of the ecology of animal movement. *Science (New York, N.Y.)*, 375 (6582), eabg1780.

Newson, P., and Krumm, J., 2009. Hidden Markov map matching through noise and sparseness. *Proceedings of the 17th ACM SIGSPATIAL international conference on advances in geographic information systems*, 336–343.

Oh, G., Leblanc, D.J., and Peng, H., 2022. Vehicle energy dataset (VED), a large-scale dataset for vehicle energy consumption research. *IEEE Transactions on Intelligent Transportation Systems*, 23 (4), 3302–3312.

Qu, L., *et al.*, 2023. HMM-based map matching and spatiotemporal analysis for matching errors with taxi trajectories. *ISPRS International Journal of Geo-Information*, 12 (8), 330.

Quddus, M.A., Ochieng, W.Y., and Noland, R.B., 2007. Current map-matching algorithms for transport applications: state-of-the art and future research directions. *Transportation Research Part C: Emerging Technologies*, 15 (5), 312–328.

Rezaei-Dastjerdehei, M.R., Mijani, A., and Fatemizadeh, E., 2020. Addressing imbalance in multi-label classification using weighted cross entropy loss function. *2020 27th national and 5th international Iranian conference on biomedical engineering (ICBME)*, 333–338.

Rossi, L., *et al.*, 2021. Human trajectory prediction and generation using LSTM models and GANs. *Pattern Recognition*, 120, 108136.

Sharath, M.N., Velaga, N.R., and Quddus, M.A., 2019. A dynamic two-dimensional (D2D) weight-based map-matching algorithm. *Transportation Research Part C: Emerging Technologies*, 98, 409–432.

Siła-Nowicka, K., *et al.*, 2016. Analysis of human mobility patterns from GPS trajectories and contextual information. *International Journal of Geographical Information Science*, 30 (5), 881–906.

Singh, J., *et al.*, 2019. Evaluating the performance of map matching algorithms for navigation systems: an empirical study. *Spatial Information Research*, 27 (1), 63–74.

Tremblay, J., *et al.*, 2018. Training deep networks with synthetic data: bridging the reality gap by domain randomization. *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR) workshops*, 969–977. https://github.com/tensorflow/models/tree/.

Vaswani, A., *et al.*, 2017. Attention is all you need. *Proceedings of the 31st international conference on neural information processing systems (NIPS'17)*, 6000–6010.

Velaga, N.R., Quddus, M.A., and Bristow, A.L., 2012. Improving the performance of a topological map-matching algorithm through error detection and correction. *Journal of Intelligent Transportation Systems*, 16 (3), 147–158.

Wan, Z., *et al.*, 2022. A long short-term memory-based approach for detecting turns and generating road intersections from vehicle trajectories. *Sensors*, 22 (18), 6997.

Wan, Z., Dodge, S., and Bohrer, G., 2023. Leveraging similarity analysis to understand variability in movement behavior. *Transactions in GIS*, 27 (5), 1441–1466.

Wan, Z., and Dodge, S., 2023. A generative trajectory interpolation method for imputing gaps in wildlife movement data. *Proceedings of the 1st ACM SIGSPATIAL international workshop on AI-driven spatio-temporal data analysis for wildlife conservation*. ACM, 1–8.

Weiss, K., Khoshgoftaar, T.M., and Wang, D.D., 2016. A survey of transfer learning. *Journal of Big Data*, 3, 1–40.

White, C.E., Bernstein, D., and Kornhauser, A.L., 2000. Some map matching algorithms for personal navigation assistants. *Transportation Research Part C: Emerging Technologies*, 8 (1–6), 91–108. www.elsevier.com/locate/trc.

Yang, R., Zhan, X., and Huang, J., 2021. Characterization and mitigation of multipath fading on multi-frequency GNSS signals in urban environment. *Aerospace Systems*, 4 (1), 19–27.

Zhang, S., *et al.*, 2022. Extended vehicle energy dataset (eVED): an enhanced large-scale dataset for deep learning on vehicle trip energy consumption. arXiv preprint arXiv:2203.08630. http://arxiv.org/abs/2203.08630.

Zhao, L., *et al.*, 2003. An extended Kalman filter algorithm for integrating GPS and low cost dead reckoning system data for vehicle performance and emissions monitoring. *Journal of Navigation*, 56 (2), 257–275.

Zhuang, F., *et al.*, 2021. A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109 (1), 43–76.